

The Secrets of Penalty Functions on Edge-preserving Image Smoothing

Xiqun Lu

College of Computer Science and Technology, Zhejiang University
China
xqlu@zju.edu.cn

Abstract

Recently, penalty functions are applied to the optimization-based edge-preserving image smoothing problem. But, to select appropriate parameter(s) for a penalty function to achieve desired smoothing effect is nontrivial. In this paper, we study the influence of penalty function on the performance of the edge-preserving image smoothing from the shrinkage rule perspective. We find some interesting secrets about penalty function on the edge-preserving image smoothing. Firstly, the shrinkage rule of a penalty function can help to determine its appropriate parameter(s). Secondly, it is more important is to select appropriate parameters for a penalty function than to select a specific penalty function, because different penalties may have very similar shrinkage rules by changing their parameter(s). Thirdly, a simple optimization framework with an appropriate parameter(s) setting of a penalty function can achieve comparable smoothing effects to those produced by much complex algorithms. To cope with the difficulties brought by nonconvex penalties, we propose an iterative algorithm by virtue of local quadratic approximation.

CCS Concepts

• Computing methodologies → Computational photography;

Keywords

edge-preserving smoothing, penalty, shrinkage rule, nonconvex, optimization

ACM Reference Format:

Xiqun Lu. 2024. The Secrets of Penalty Functions on Edge-preserving Image Smoothing. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3698017>

1 Introduction

The edge-preserving image smoothing is a valuable tool for a variety of applications in computer graphics and image processing, and has attracted continuous attention from research community over the last few decades.

As pointed out by [Farbman et al. 2008], the edge-preserving image smoothing is to achieve a compromise between two possibly

conflicting goals, thus, it is natural to formulate it in an optimization problem,

$$\min_{\mathbf{u}} \{D(\mathbf{u}, \mathbf{f}) + \lambda R(\mathbf{u})\}, \quad (1)$$

where the data fidelity term $D(\mathbf{u}, \mathbf{f})$ is to ensure that the smoothed output $\mathbf{u}$ is as close as possible to the input image $\mathbf{f}$, while the regularization term $R(\mathbf{u})$ is to impose some expected properties on $\mathbf{u}$, such as, $\mathbf{u}$ has sparse gradients. But sometimes, the regularization term $R(\mathbf{u})$ is not always necessary, e.g. the bilateral filter (BLF) [Tomasi and Manduchi 1998].

Recently, a variety of penalty functions have been applied on the regularization term, such as the Welsch penalty function used in the SD filter [Ham et al. 2018], and the generalized Charbonnier penalty used in [Liu et al. 2020]. Or on both the data fidelity and regularization terms, such as [Liu et al. 2022], in which they modified the Huber penalty [Huber 1992] to the truncated Huber. But, to find appropriate parameter(s) for a penalty function is still unsolved. The common way is to tune the parameter(s) of a specific penalty for each input image individually.

Since the goal of the edge-preserving smoothing is very close to that of the shrinkage function, in this paper, we study the shrinkage rules of penalty functions. The shrinkage rules can explain intuitively why some penalty functions can achieve desired smoothing effects, while others cannot. The shrinkage rules of a penalty function also can help us to determine appropriate parameter(s) for a penalty function even before running algorithms, because the smoothing effect is predictable by the shrinkage rules, but not depended on the input image. In order to study the influence of penalty function on the performance of the edge-preserving image smoothing, we propose a simple edge-preserving optimization framework, in which an adopted penalty is only applied on the regularization term. However, the adopted penalty could be non-convex and non-differentiable, we propose an iterative algorithm based on local quadratic approximation.

We find some interesting secrets about penalty functions on edge-preserving image smoothing:

- The shrinkage rules of a penalty function can help to determine its appropriate parameter(s), which can free us from tedious labor to tune parameter(s) for each image individually.
- Selecting appropriate parameter(s) for a penalty is much important than searching a suitable penalty for edge-preserving image smoothing, since different penalties may have very similar shrinkage rules by changing their parameter(s).
- A simple optimization framework with an appropriately penalized regularization term can achieve comparable smoothing effects as those produced by much more complex algorithms.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs International 4.0 License.

SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1140-4/24/12
<https://doi.org/10.1145/3681758.3698017>

2 The Shrinkage Rules of Various Penalty Functions

Penalty function plays an important role in regularizing the output for the edge-preserving image smoothing. But, the problem of how to select an appropriate penalty function or how to select suitable parameter(s) for a specific penalty function so as to achieve desired smoothing output is still unsolved. Additionally, a variety of penalty functions may seem to be so different, which would make the selection even more hard.

In order to better understand penalty function, it is informative to consider the scalar case of the following shrinkage problem:

$$s(y) = \underset{x}{\operatorname{argmin}} \left\{ \frac{1}{2} \|x - y\|_2^2 + \phi(x) \right\}, \quad (2)$$

The function $s(y)$ is called the shrinkage function because it may shrink the small values of y toward zero. The exact form of $s(y)$ depends on the penalty function $\phi(x)$.

In the Fig.1, we illustrate the shrinkage rules of a variety of penalty functions with different parameter(s). Please refer to Table I in the supplement for the definitions of these penalty functions and their first-order derivatives.

Since in [Farbman et al. 2008], the penalty function is not given explicitly, we deduce the penalty function from the weighting function. When the parameter a takes small values, the WLS penalty almost has no shrinkage ability, such as the red line and the red dashed line in Fig.1(a). When $a = 1$, the shrinkage of WLS penalty is just the soft threshold function $\max(|y| - 1, 0) \operatorname{sign}(y)$ (see the green line in Fig.1 (a)). The soft threshold function can force small values to zero directly, but it will shift large values with a bias (here the bias quantity is 1), and this bias will introduce the “color shift” artifacts in the smoothed output, for the details, please refer to Sec.4. When the parameter a is increasing (the dashed green curve and the blue curve in Fig.1(a)), the bias for large values is becoming smaller. When $a = 2$, the shrinkage function is like a hyperbola (the blue curve in Fig.1 (a)) separated by the black dashed dot line that corresponding to no shrinkage.

In Fig.1 (b), the Welsch penalty [Ham et al. 2018] has more strong shrinkage function when the parameter a takes smaller values. When $a = 0.1$, it has a larger bias. One property of the Welsch penalty is that it cannot threshold small values to zero directly. It could explain why the smoothed results of the Welsch penalty will look much sharper than those produced by the WLS penalty.

In Fig.1 (c), we can see that when the parameter a takes 0.1, the Huber penalty [Huber 1992] is just the soft threshold function $\max(|y| - 1, 0) \operatorname{sign}(y)$ (see the red line in Fig.1 (c)). For the Huber penalty, no matter how the parameter a is set, the shrinkage rule keeps the same for large values, and always has a bias shift “1” from large values. This explains why the smoothed outputs produced by the Huber penalty always have obvious “color shift” artifacts.

When the parameter a takes small values, the shrinkage rule of the generalized Charbonnier penalty [Liu et al. 2020] is highly nonlinear (the red curve and the dashed red curve in Fig.1 (d)), and also has strong ability to set small input values to zero. When $a = 1$, it is the soft threshold function $\max(|y| - 1, 0) \operatorname{sign}(y)$. When a becomes larger, it has stronger shrinkage function, but it will lose

its ability to threshold small values to zero, and brings more obvious “color shift” artifacts to the smoothed output.

In addition to these penalty functions which are controlled only by one parameter, next we introduce two penalty functions which have two parameters a and b . For this two penalties we draw two shrinkage rules for each of them by alternatively fixing one parameter and changing the other one.

The truncated Huber was designed in [Liu et al. 2022]. From Fig.1 (c),(e) and (f), we can see that the difference between the truncated Huber and the Huber is that the truncated Huber forces to keep large values untouched, and this makes the truncated Huber more appropriate for edge-preserving image smoothing. The parameter a in the truncated Huber controls the shape of shrinkage function for small values, while the parameter b controls how quickly the penalty drops off for large values.

Since the smoothly clipped absolute deviation (SCAD) in [Fan and Li 2001] is designed only for positive values, here we extend it to the whole real values. The modified SCAD penalty and its derivative can be found in the Table I in the supplement. The modified SCAD penalty corresponds to a quadratic spline function with knots at a and ab . From Fig.1 (g) and (h), we can see that when the modified SCAD penalty has stronger ability of shrinkage when increasing the parameter a , while the large values are always kept untouched at the same time. Compared with the truncated Huber, the modified SCAD leaves large values not excessively penalized and makes the solution continuous, and is less sensitive to the parameters settings (refer to Fig.1 (g) and (h)).

3 A Simple Unified Edge-preserving Image Smoothing Framework

Many edge-preserving image smoothing algorithms integrate different penalty functions into the regularization term, or into both the data fidelity and regularization terms of the edge-preserving optimization problem shown in Eq.1, e.g. [Farbman et al. 2008; Ham et al. 2018; Liu et al. 2020, 2022; Paris et al. 2015]. Conventionally, to penalize the data fidelity in a quadratic way is enough, and we are primarily interested in the influence of penalty function on the sparsity of the output’s gradients, hence we simply formulate the edge-preserving image smoothing problem as:

$$E(\mathbf{u}) = \min_{\mathbf{u}} \left\{ \frac{1}{2} \|\mathbf{u} - \mathbf{f}\|_2^2 + \lambda \phi(\nabla \mathbf{u}) \right\}, \quad (3)$$

where the parameter λ controls the smoothing strength, the larger λ , the smoother output will be. The main difficulty to minimize the problem in Eq.3 directly is due to the penalty function ϕ , because it is generally nonconvex and non-differentiable. Instead of minimizing $E(\mathbf{u})$ directly, we use the majorization-minimization (MM) approach to solve a sequence of simpler minimization problems:

$$\mathbf{u}^{k+1} = \underset{\mathbf{u}}{\operatorname{argmin}} \left\{ \frac{1}{2} \|\mathbf{u} - \mathbf{f}\|_2^2 + \lambda G_k(\mathbf{u}) \right\}, \quad (4)$$

which requires that each function $G_k(\mathbf{x})$ to be an upper bound of $\phi(\mathbf{x})$ at $\mathbf{x} = \mathbf{x}^k$. That is,

$$\begin{aligned} G_k(\mathbf{x}) &\geq \phi(\mathbf{x}) \quad \forall \mathbf{x} \\ G_k(\mathbf{x}^k) &= \phi(\mathbf{x}^k) \end{aligned} \quad (5)$$

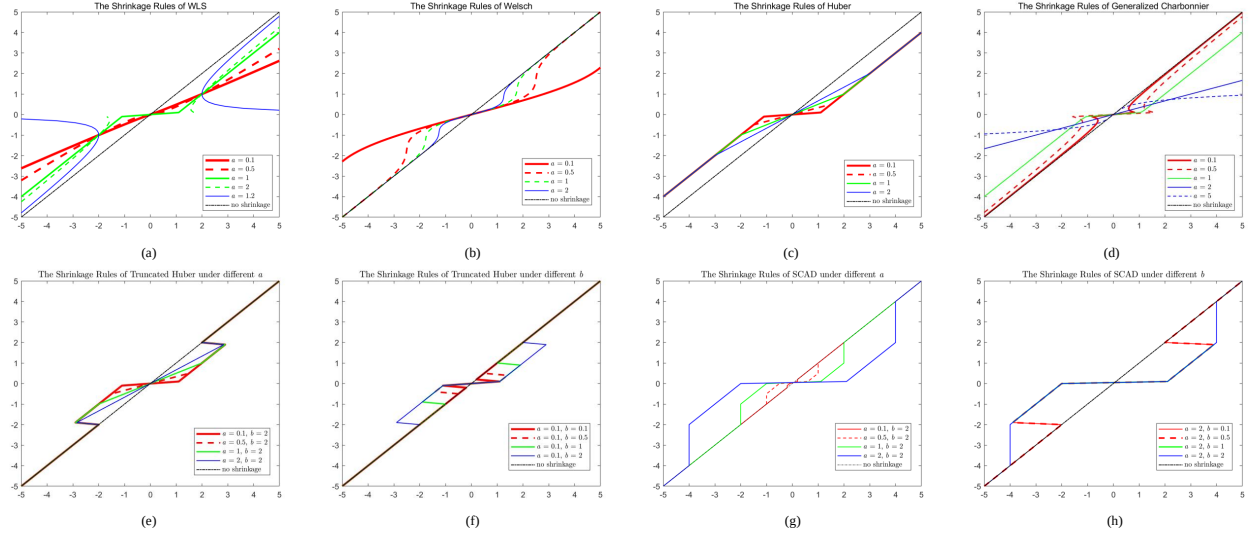


Figure 1: From left to right, and up to down are the shrinkage rules of the WLS penalty [Farbman et al. 2008], the Welsch penalty [Ham et al. 2018], the Huber penalty [Huber 1992], the generalized Charbonnier penalty [Liu et al. 2020], the truncated Huber [Liu et al. 2022] with different a settings, the truncated Huber with different b settings, the modified SCAD penalty with different a settings, and the modified SCAD penalty with different b settings, respectively

To make the explanation simple and easy, it is useful to consider first the scalar case. Specifically, the quadratic function $G_k(x)$ has the form $G_k(x) = mx^2 + c$. Using the scalar versions of the two conditions in Eq.5, and with simple arithmetic, we can obtain the quadratic function $G_k(x)$ as

$$G_k(x) = \frac{\phi'(x^k)}{2x^k} x^2 + \phi(x^k) - \frac{x^k}{2} \phi'(x^k) \quad (6)$$

The prime means the derivative of $\phi(x)$ w.r.t. x . With this function $G_k(x)$, we have $G_k(x) \geq \phi(x)$ for all x , and $G_k(x)$ is agree with $\phi(x)$ when $x = x^k$. Please note that the coefficient of quadratic term is the edge stopping function of penalty $\phi(x)$. The second and third terms in Eq.6 are constant terms, so neglecting them would not affect the optimization result.

This scalar upper bound function can easily be extended to the vector form in Eq.4. Given a previous estimation $\mathbf{u}^k$, the next estimation will be:

$$\mathbf{u}^{k+1} = \left(\mathbf{I} + \lambda \left(\partial_x^T \mathbf{W}_x \partial_x + \partial_y^T \mathbf{W}_y \partial_y \right) \right)^{-1} \mathbf{f} = (\mathbf{I} + \lambda \mathbf{L})^{-1} \mathbf{f} \quad (7)$$

where

$$\begin{aligned} \mathbf{W}_x &= \text{diag} \left(\frac{\phi'(\partial_x \mathbf{u}_1^k)}{\partial_x \mathbf{u}_1^k} \dots \frac{\phi'(\partial_x \mathbf{u}_N^k)}{\partial_x \mathbf{u}_N^k} \right) \\ \mathbf{W}_y &= \text{diag} \left(\frac{\phi'(\partial_y \mathbf{u}_1^k)}{\partial_y \mathbf{u}_1^k} \dots \frac{\phi'(\partial_y \mathbf{u}_N^k)}{\partial_y \mathbf{u}_N^k} \right) \end{aligned} \quad (8)$$

Here $\mathbf{I}$ denotes an identity matrix and N is the number of pixels in input image, the superscript k denotes the iteration number. $\mathbf{L}$ in Eq.7 is a five-point spatially inhomogeneous Laplacian matrix.

We can see that many edge-preserving image smoothing algorithms can be considered as special or extended cases of this unified formulation. In [Farbman et al. 2008], the penalty function is chosen as $(2-a)|x|^{2-a}$. In the SD filter [Ham et al. 2018], if the weights from the static guidance image are not taken into the regularization term, and to use the Welsch penalty, then the SD filter is reduced to the form of Eq.7. In [Liu et al. 2020], they use an obscurity additive half-quadratic minimization method [Nikolova and Ng 2005], the solution can also be reduced to Eq.7 with the weighting diagonal matrices are the edge stopping functions of the generalized Charbonnier. In [Liu et al. 2022], the penalty function was not only imposed on the regularization term, but on the data fidelity term as well. They still use the additive half-quadratic minimization method to transform the nonconvex truncated Huber function into a quadratic form, from Eq.12 in [Liu et al. 2022], we can see that the weights are corresponding to the edge stopping function of the truncated Huber penalty. Compared with these algorithms, the derivation in this paper is much simple and direct, and the weights are directly related to the edge stopping function of the adopted penalty function.

4 Experimental Results

In this part, we evaluate the influence of penalty function on the performance of edge-preserving image smoothing in the simple optimization framework described above. In order achieve strong smoothing effects, while set the iteration number to 1, in the experiment, we set the parameter λ to a large value, such as 500.

For the WLS penalty [Farbman et al. 2008], if we set the parameter $a = 1.2$ (Fig.2 (b)), the smoothed image has “color shift” artifacts, e.g., the central yellow parts on the flower. But, when a is set as 2

(Fig.2 (c)), the output has not such obvious “color shift” artifact. This phenomenon is due to the fact that the shrinkage rule of the WLS penalty when $a = 2$ has less bias than the rule when a takes 1.2 does. Please refer to the dashed green curve and the blue curve in Fig.1 (a). For the generalized Charbonnier penalty [Liu et al. 2020], if a is getting larger, the output is becoming more and more smooth, and more “color shift” artifacts will occur (Fig.2 (d)). This is expected because the shrinkage curves of the generalized Charbonnier penalty have a great bias from large values when $a \geq 1$. Please refer to green and blue lines in Fig.1 (d).



Figure 2: The image smoothing results produced by some penalty functions. In the experiment, we set the iteration number as 1, and the parameter λ as 500.

For the Welsch penalty [Ham et al. 2018], with the smaller a , the output becomes smoother, and more “color shift” artifacts will appear. One property of the Welsch penalty is that it cannot set small gradients to zero directly. It can only shrink small gradients, and the shrinkage degree depends on its slope around the zero. So the output looks more sharper for some details than those produced by the WLS penalty and the Charbonnier penalty.

Both the truncated Huber [Liu et al. 2022] and the modified SCAD have two parameters. From Fig.1 (e), we can find that when $a = 1$ and $b = 2$, the truncated Huber can achieve a good balance between shrinkage and bias. Since the shape of the shrinkage curve of the modified SCAD is very similar to that of the truncated Huber under this parameters setting, it is hard to tell the difference between the results. To save the space, we only present the result of the modified SCAD in Fig.2 (f). The shape of the shrinkage curve with this setting is also very close to the Welsch’s shrinkage curve

when $a = 1$, but a little sharper. As a result, the modified SCAD’s output is also sharper than that of the Welsch penalty, please look at the Fig.2 (e) and (f) carefully.

From the above experimental results, we find that the same penalty function may produce very different outputs when using different parameter(s) settings, whereas different penalty functions may produce outputs which have very close visual qualities. We owe this phenomenon to the fact that the shrinkage rules of penalty functions are changing with parameter(s). The shrinkage rule is independent of the input image, as a consequence, we need not to waste time to tune the parameter(s) for each image individually. All we need to do is to look at the shrinkage properties of a specific penalty under different parameter(s) settings, and then select an appropriate parameter(s) setting for an application. In another example, we investigate the convergent properties of different penalties by setting the parameter λ to a very large value, such as 50000, for the details, please check Fig.3 in the supplement.

5 Conclusions

In this paper, we study the effect of penalty function on the edge-preserving image smoothing from the shrinkage perspective. We find that the shrinkage rules of a penalty function do affect the performance of the edge-preserving image smoothing. A penalty may lead to very different shrinkage rules by setting different parameter(s), and different penalty functions may have similar shrinkage rules. This phenomenon could explain the phenomenon why some different penalty functions can produce very similar outputs. The shrinkage rules are independent of the input image, thus it will help us to select appropriate parameter(s) for an application, and no need to tune the parameter(s) for each image separately. The shrinkage rules also provide a possibility to design edge-aware filters as done in [Paris et al. 2015], but in a more theoretical way.

References

- Jianqing Fan and Runze Li. 2001. Variable Selection via Nonconcave Penalized Likelihood and its Oracle Properties. *J. Amer. Statist. Assoc.* 96, 456 (2001), 1348–1360. <https://doi.org/10.1198/016214501753382273> arXiv:https://doi.org/10.1198/016214501753382273
- Zeev Farbman, Raanan Fattal, Dani Lischinski, and Richard Szeliski. 2008. Edge-preserving decompositions for multi-scale tone and detail manipulation. *ACM Trans. Graph.* 27, 3 (2008), 67. <https://doi.org/10.1145/1360612.1360666>
- Bumsub Ham, Minsu Cho, and Jean Ponce. 2018. Robust Guided Image Filtering Using Nonconvex Potentials. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40, 1 (2018), 192–207. <https://doi.org/10.1109/TPAMI.2017.2669034>
- Peter J. Huber. 1992. *Robust Estimation of a Location Parameter*. Springer New York, New York, NY, 492–518. https://doi.org/10.1007/978-1-4612-4380-9_35
- Wei Liu, Pingping Zhang, Xiaolin Huang, Jie Yang, Chunhua Shen, and Ian Reid. 2020. Real-time Image Smoothing via Iterative Least Squares. *ACM Trans. Graph.* 39, 3, Article 28 (jun 2020), 24 pages. <https://doi.org/10.1145/3388887>
- Wei Liu, Pingping Zhang, Yinjie Lei, Xiaolin Huang, Jie Yang, and Michael Ng. 2022. A Generalized Framework for Edge-Preserving and Structure-Preserving Image Smoothing. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 10 (2022), 6631–6648. <https://doi.org/10.1109/TPAMI.2021.3097891>
- Mila Nikolova and Michael K. Ng. 2005. Analysis of Half-Quadratic Minimization Methods for Signal and Image Recovery. *SIAM Journal on Scientific Computing* 27, 3 (2005), 937–966. <https://doi.org/10.1137/030600862> arXiv:https://doi.org/10.1137/030600862
- Sylvain Paris, Samuel W. Hasinoff, and Jan Kautz. 2015. Local Laplacian filters: edge-aware image processing with a Laplacian pyramid. *Commun. ACM* 58, 3 (feb 2015), 81–91. <https://doi.org/10.1145/2723694>
- C. Tomasi and R. Manduchi. 1998. Bilateral filtering for gray and color images. In *Sixth International Conference on Computer Vision (IEEE Cat. No.98CH36271)*. 839–846. <https://doi.org/10.1109/ICCV.1998.710815>